面向实时动态调度的 Guarded EOH-Go 启发式代码进化研究

EOH-Go 实验工作区整理稿

2026 年 4 月 26 日

摘要

封闭园区配送中的动态取送货问题要求系统在新请求到达后快速给出可行路线，同时尽可能降低最终调度成本。参考论文《实时路由与并发计算在封闭商业园区配送服务中的应用》将该类问题建模为具有时间窗约束的动态取送货问题，并以响应时间 Res. 与目标函数值 J 共同评价实时路由算法。本文沿用这一评价口径，但研究对象从人工设计的实时路由求解器转向大语言模型驱动的启发式代码自动进化：我们使用 Evolution of Heuristics (EOH) 从 SA 基线的 Go 插入函数出发，自动生成和变异 `InsertShips` 代码，并通过 Go 编译、动态仿真和 evaluator guard 对候选代码进行筛选。

最终实现的系统是一个小型 Guarded EOH-Go pipeline：SA seed、LLM mutation、Go code extraction、Go compile、dynamic source evaluation、candidate guard、filtered-best selection 和 cleaned reporting。实验覆盖 RC101–RC105、三种动态密度 $d \in \{d25, d50, d75\}$ 以及五种请求到达节奏 $t \in \{1.0, 0.9, 0.8, 0.7, 0.6\}$，共 75 个 cell。清洗后得到 43 个有效 SA–EOH 对比，其中 EOH 改善 16 个、持平 11 个、变差 16 个；另有 32 个 cell 因无有效输出、负值或 suspicious-low 结果被排除。结果表明，EOH-Go 不能被表述为全面优于 SA，但在部分密度和到达节奏下能够发现有效的 Go 插入启发式；同时，guard/filter 是 LLM 代码进化进入动态调度问题时不可省略的可信评估层。

关键词： 动态车辆路径；实时调度；Evolution of Heuristics；大语言模型；Go 代码进化；响应时间；evaluator guard

1 引言

动态车辆路径问题与静态车辆路径问题的核心差异在于，请求并不会在求解开始时全部已知，而是随着时间推进逐渐揭示。参考论文讨论的封闭商业园区配送场景同时包含快递包裹配送、上门收件和食品配送等服务类型，其中一部分请求具有严格时间窗，并且只有在服务过程中才会出现。对于这类系统，算法评价不能只看最终路径成本，还必须考虑从新请求到达到系统给出新路线之间的处理时间。参考论文因此将响应时间定义为系统状态发生变化后求解器生成相应新解所消耗的时间，并在实验表格中同时报告 Res. 与 J。

本文的出发点与该参考论文一致：实时动态调度算法必须在质量与响应速度之间取得平衡。不同的是，参考论文主要研究人工设计的实时路由求解器、插入任务、优化任务、自适应内存和并发计算框架，而本文进一步追问：在已有 Go 调度程序和 SA 插入基线的基础上，大语言模型是否能够自动进化出更好的插入启发式代码？换言之，本文关注的不是重新设计整个实时路由系统，而是把实时系统中新请求到达时最关键的插入函数 `InsertShips` 作为代码进化目标，让 LLM 在 EOH 框架下生成、变异、编译并评估候选 Go 代码。

这一路线的吸引力在于，动态路由问题高度依赖场景，手工设计一个在所有密度、时间节奏和实例上都表现稳定的插入启发式非常困难。参考论文也强调不同动态比例会对应不同类型的园区场景，因此算法表现需要分场景比较。EOH 提供了一种可能性：不预设唯一的通用启发式，而是在给定数据分布和评价器下自动搜索可执行代码。然而，这种方法也带来新的风险。LLM 生成的是可执行程序，而不是受限的参数向量；如果评价器存在漏洞，进化过程可能主动利用漏洞，例如漏单、提前退出、输出异常低成本或触发解析错误。因此，本文最终实现的重点不是“让 LLM 写一段看似聪明的代码”，而是建立一条可复现、可过滤、可审计的 Guarded EOH-Go 实验链路。

本文贡献可以概括为三点。第一，将 EOH 从常见的 Python 或伪代码启发式生成迁移到 Go 实时动态调度代码生成，评价对象是可编译、可运行的 `InsertShips` 函数。第二，提出并实现 Guarded EOH-Go pipeline，把 LLM mutation、Go 编译、动态仿真、异常候选过滤和 cleaned reporting 串联起来。第三，沿用参考论文的 Res./J 双指标与动态密度实验思想，在 RC101–RC105 的动态 Solomon-style 数据源上分析 EOH 在密度 d 与到达节奏 t 下的局部有效性、稳定性和场景敏感性。

2 相关工作与参考问题设定

2.1 实时动态取送货路由

参考论文研究的是封闭商业园区内的混合配送服务，并将其形式化为带时间窗约束的动态取送货问题。该问题可被视为动态车辆路径问题的一个复杂子类，其中请求集合随时间变化，而车辆、地理节点和边集保持相对固定。论文中的目标函数 J 衡量整个动态过程中累计的路线成本，响应时间则衡量系统从新状态出现到生成新路线的速度。该评价口径对本文非常关键，因为 EOH 进化出的插入代码即使降低了 J，如果响应时间不可接受，也未必适合实时调度。

参考论文还将 Solomon benchmark 修改为不同动态比例的数据源，例如 25%、50%、75% 的请求作为预先已知请求，其余作为动态请求。这一点直接启发了本文的密度变量设计。本文使用 `solomon_benchmark_d25`、`solomon_benchmark_d50` 和 `solomon_benchmark_d75` 作为动态密度源，并进一步引入 arrival scale t 来改变请求释放节奏。这样做的目的不是简单增加参数，而是让实验真正反映不同动态场景对插入启发式的影响。

2.2 EOH 与 LLM 辅助算法设计

Evolution of Heuristics 将大语言模型与进化计算结合，用自然语言描述和可执行代码共同表示启发式，并通过评价器对候选进行选择和变异。与一次性 prompt 生成算法相比，EOH 的核心在于 population、mutation 和 selection 构成的迭代搜索过程。FunSearch、LLM4AD、ReEvo 等工作也表明，LLM 生成程序与自动评价器结合后可以成为算法设计工具。

但是，这类方法的共同前提是评价器必须可信。对于数学构造或封闭任务，评价函数通常较容易检查；对于实时调度 Go 程序，候选代码可能通过非预期方式降低表面成本，例如不插入某些订单、遇到不可行情况直接 break、返回未更新的 dispatch 或制造异常输出。因此，本文的最终实现聚焦于 EOH 主线中的工程可信层：compile/run、guard、logging 和 filtered reporting。这个选择与最终实验一致：当前论文的重点是 Guarded EOH pipeline 本身，即如何让 LLM 生成的 Go 插入代码经过真实编译、动态仿真和可信过滤后再进入主表。

3 问题定义与实验对象

本文的目标函数是 Go 调度程序中的 `InsertShips`。当动态请求到达时，该函数接收当前 dispatch、请求起点集合、请求终点集合和订单数量，返回更新后的 dispatch。SA baseline 使用项目中已有的 Go 插入逻辑。EOH-Go 以该基线函数作为 seed，让 LLM 生成变异版本，然后将候选代码替换到 Go 项目中进行编译和动态仿真。

实验变量包括两类。第一类是动态密度 d，取值为 $d25, d50, d75$，分别对应不同动态比例的数据源。第二类是到达节奏 t，取值为 $1.0, 0.9, 0.8, 0.7, 0.6$。早期尝试中曾使用不直接改变请求释放节奏的时间参数，导致同一密度下 J 几乎不变；最终实现采用 arrival scale，使 t 真正影响动态订单出现节奏。该设计与参考论文中的 timescale/hyperparameter 分析在思想上对应：动态系统不仅要比较算法，还要观察时间尺度和场景强度如何改变算法表现。

评价指标沿用参考论文风格。Res. 表示首次产生可行 dispatch 的响应时间，J 表示最终路线成本。主比较采用 $\Delta J = J_{EOH} - J_{SA}$：当 $\Delta J < 0$ 时，EOH 的路线质量优于 SA；当 $\Delta J = 0$ 时视为持平；当 $\Delta J > 0$ 时视为变差。由于实时系统还要求响应速度，本文同时报告 $\Delta Res.$，但不把所有结论压缩为单一标量。

4 方法：Guarded EOH-Go Pipeline

最终实现的 pipeline 如下：

$$\text{SA seed} \rightarrow \text{LLM mutation} \rightarrow \text{Go extraction} \rightarrow \text{Go compile}$$
$$\rightarrow \text{dynamic simulation} \rightarrow \text{candidate guard} \rightarrow \text{filtered-best report}$$

这一流程中的每一步都有明确作用。SA seed 保证搜索从一个可运行的基线开始；LLM mutation 生成新的 `InsertShips` 候选；Go extraction 负责从 LLM 输出中提取函数代码；Go compile 将候选放回真实项目中检验语法和依赖；dynamic simulation 在 d, t 对应的动态源上运行调度过程；candidate guard 对候选进行有效性过滤；最终报告只使用 filtered-best，而不使用 raw-best。

4.1 候选过滤规则

guard 的设计来自实际实验中暴露出的风险。早期运行曾出现候选全部被评为 10^9 的惩罚值，也出现过 J 异常极小的候选。前者通常对应编译、运行或解析失败；后者更危险，因为它可能在表面上优于 SA，但实际原因是漏单或提前退出。因此最终主表采用以下清洗规则：若 SA J 缺失，记为 `excluded_no_sa`；若 EOH J 缺失，记为 `excluded_no_eoh`；若 EOH $J < 0$，记为 `excluded_negative_eoh`；若 EOH $J < 0.3\times$ SA J，记为 `excluded_suspicious_low`。其余候选再按 ΔJ 分类为 improved、tie 或 worse。

这种清洗并不是为了人为剔除不利结果，而是为了使 LLM 代码进化具备最基本的可信性。对于人工编写的固定启发式，算法不会主动寻找评价器漏洞；对于 LLM 进化程序，评价器漏洞本身会成为搜索目标。因此，guard/filter 是本文最重要的方法组件之一，也是本文与普通路由算法比较论文的区别。

5 实验设计

实验覆盖 RC101–RC105 共五个 Solomon-style 实例，每个实例在 $d25, d50, d75$ 三种动态密度下，分别测试 $t = 1.0, 0.9, 0.8, 0.7, 0.6$ 五种到达节奏，共 75 个 cell。EOH 使用 deepseek-v4-flash 模型，每个 cell 设置 generations=1、pop_size=4、evaluation timeout=120s、run timeout=60s，并使用 $J + 0.2 \times Res.$ 的复合 objective 引导候选选择。需要强调的是，复合 objective 用于进化过程中的候选排序，而论文主表仍分别报告 Res. 与 J，以保持与参考论文的评价口径一致。

数据处理分为原始记录和清洗记录两层。原始记录保留每个 cell 的 population、candidate id、raw objective、valid/suspicious/invalid candidate 数量和运行输出；清洗记录只用于论文主表。这样既能避免异常值污染主结论，又能在需要时回溯候选代码和失败原因。

6 实验结果

6.1 总体结果

在 75 个 cell 中，guard 清洗后得到 43 个有效对比，另有 32 个 cell 被排除。其中有效对比的分布为：EOH 改善 16 个，持平 11 个，变差 16 个。被排除结果中，`excluded_no_sa` 为 10 个，`excluded_no_eoh` 为 10 个，`excluded_suspicious_low` 为 10 个，`excluded_negative_eoh` 为 2 个。这说明 EOH-Go 已经能够产生一批有效候选，但同时也暴露出自动代码进化的不稳定性。

从有效 cell 看，最强单次改善出现在 RC105 的 d50，$t = 0.9$，SA $J = 476.28$，EOH $J = 302.13$，$\Delta J = -174.15$。不过，单次最强改善不应被直接写成稳定优势，因为后续 repeat validation 显示部分强胜点会退化。因此本文的主结论是局部有效和场景敏感，而不是全局支配。

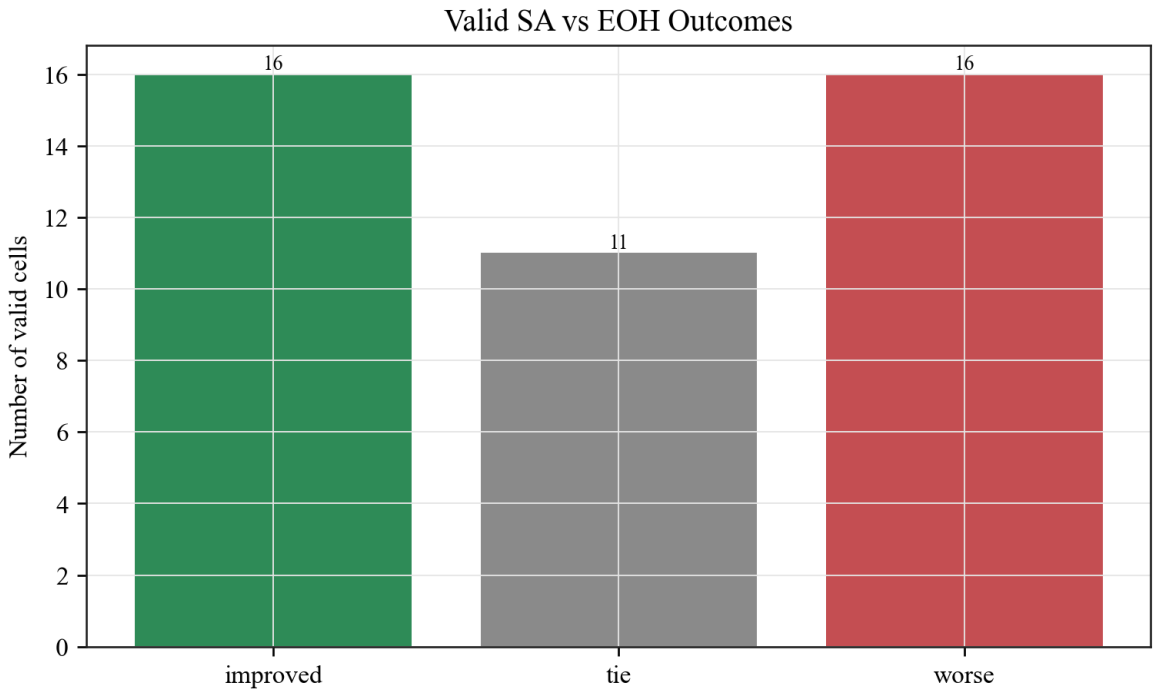

图 1: Guard 清洗后的总体结果计数。EOH 在有效 cell 中既有改善也有变差，说明该方法更适合被表述为场景相关的自动启发式设计，而不是通用替代 SA。

6.2 主表：有效 SA–EOH 对比

图 2 给出了所有有效 cell 的论文风格对比表。表中同时报告 SA 与 Guarded EOH 的 Res. 和 J，并用星号标记每行更低的 J。这种表格形式直接参考了参考论文中算法效率比较表的写法：不只比较最终质量，也保留响应时间，使读者能够看到质量改善是否伴随额外时间开销。

Table 1 Performance of SA and Guarded EOH on valid dynamic-dispatch cells

Instance	d	t	SA		Guarded EOH		ΔJ	Outcome
			Res.	J	Res.	J		
RC101	d25	1.0	0.689	664.12	0.914	**657.73***	-6.39	improved
RC101	d25	0.6	0.736	557.48	0.872	**479.50***	-77.98	improved
RC101	d50	1.0	1.085	713.52	1.625	**660.73***	-52.79	improved
RC101	d50	0.9	0.960	697.24	2.884	**652.51***	-44.73	improved
RC101	d75	1.0	1.175	**549.48***	1.232	**549.48***	0.00	tie
RC101	d75	0.9	1.291	435.48	5.035	**424.57***	-10.92	improved
RC102	d25	1.0	0.443	**642.69***	0.406	**642.69***	0.00	tie
RC102	d25	0.8	0.443	**390.12***	0.716	484.04	+93.93	worse
RC102	d25	0.7	0.525	**425.08***	0.489	**425.08***	0.00	tie
RC102	d50	1.0	1.214	**551.81***	2.538	562.47	+10.66	worse
RC102	d50	0.9	1.305	540.70	2.538	**516.26***	-24.44	improved
RC102	d50	0.8	1.333	**384.90***	2.255	401.17	+16.27	worse
RC102	d50	0.7	1.331	384.90	1.602	**282.16***	-102.74	improved
RC102	d50	0.6	1.270	**377.90***	2.852	460.28	+82.38	worse
RC102	d75	1.0	3.069	383.81	2.997	**350.27***	-33.54	improved
RC102	d75	0.7	2.807	**370.12***	2.807	**370.12***	0.00	tie
RC103	d75	1.0	18.695	**80.08***	34.896	413.70	+333.62	worse
RC103	d75	0.8	18.625	**80.08***	18.638	**80.08***	0.00	tie
RC103	d75	0.6	18.383	**80.08***	32.093	119.27	+39.19	worse
RC104	d25	1.0	0.236	**495.79***	1.213	715.88	+220.09	worse
RC104	d25	0.9	0.272	**495.79***	1.214	715.88	+220.09	worse
RC104	d25	0.8	0.257	**495.79***	0.265	**495.79***	0.00	tie
RC104	d25	0.7	0.267	**495.79***	0.254	**495.79***	0.00	tie
RC104	d25	0.6	0.272	**495.79***	0.253	**495.79***	0.00	tie
RC104	d50	0.8	0.460	281.67	1.349	**141.13***	-140.54	improved
RC104	d50	0.7	0.400	**281.67***	1.389	303.08	+21.41	worse
RC104	d50	0.6	0.424	281.67	1.807	**195.03***	-86.64	improved
RC104	d75	1.0	7.021	**119.22***	20.320	485.59	+366.37	worse
RC104	d75	0.9	8.125	**119.22***	7.699	**119.22***	0.00	tie
RC104	d75	0.8	8.834	119.22	11.681	**73.21***	-46.01	improved
RC104	d75	0.7	7.570	119.22	10.788	**73.21***	-46.01	improved
RC104	d75	0.6	8.635	119.22	12.887	**73.21***	-46.01	improved
RC105	d25	1.0	0.660	**459.84***	0.686	575.44	+115.60	worse
RC105	d25	0.9	0.500	**389.73***	0.676	590.04	+200.30	worse
RC105	d25	0.8	0.609	**377.54***	0.623	**377.54***	0.00	tie
RC105	d25	0.7	0.684	**290.81***	0.748	**290.81***	0.00	tie
RC105	d25	0.6	0.950	**290.81***	0.689	409.81	+119.00	worse
RC105	d50	1.0	0.759	**404.95***	1.664	467.46	+62.52	worse
RC105	d50	0.9	0.769	476.28	2.989	**302.13***	-174.15	improved
RC105	d75	1.0	1.216	**304.19***	4.071	331.61	+27.42	worse
RC105	d75	0.9	1.087	252.77	1.717	**170.01***	-82.76	improved
RC105	d75	0.8	1.396	**310.78***	4.963	348.29	+37.51	worse
RC105	d75	0.6	1.180	310.78	1.409	**226.78***	-84.00	improved

Note: Only cleaned valid cells are shown. * marks the lower J in each row. Delta J = EOH J - SA J; negative values indicate improvement by EOH.

图 2: 所有有效 cell 的 SA 与 Guarded EOH 对比表。星号表示每行更低的 J，负的 ΔJ 表示 EOH 改善。

6.3 密度影响

按密度统计，有效 cell 中 $d25$ 的结果为 improved=2、tie=7、worse=6；$d50$ 为 improved=7、tie=0、worse=5；$d75$ 为 improved=7、tie=4、worse=5。可以看到，EOH 的改善主要集中在 $d50$ 和部分 $d75$ cell，而在 $d25$ 中更多表现为持平或变差。这一现象与直觉一致：低密度动态请求较少，简单 SA 插入已足够有效，复杂变异不一定带来收益；中高密度下插入选择空间更大，LLM 生成的策略更可能找到不同于 SA 的改进路径。

6.4 密度–到达节奏二维影响

图 5 将所有有效 cell 的 ΔJ 放到实例、密度和到达节奏构成的网格中。该图比单个平均值更重要，因为它显示 EOH 的效果不是单调随 t 或 d 改变，而是在局部区域出现改善。参考论文同

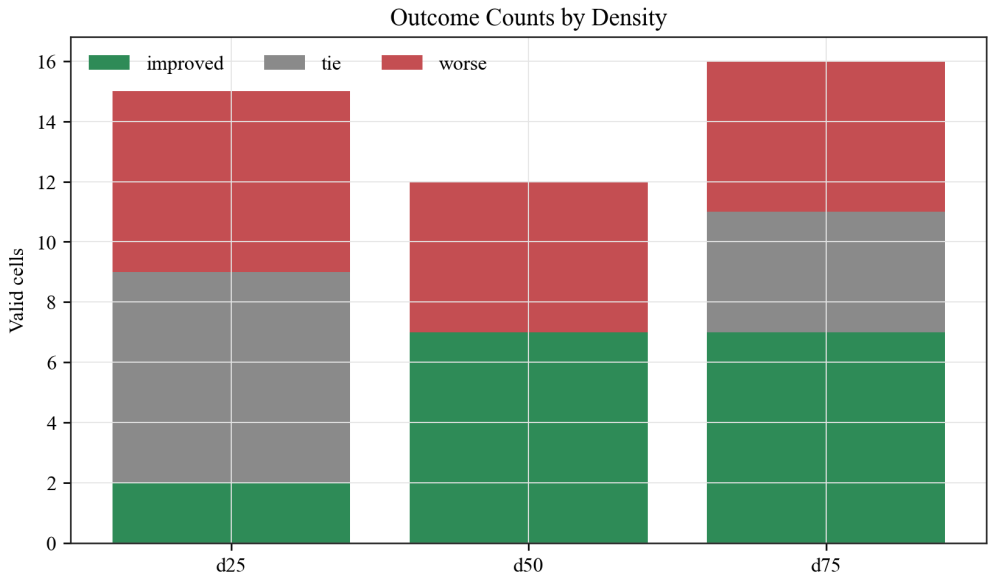

图 3: 不同动态密度下 improved/tie/worse 的分布。

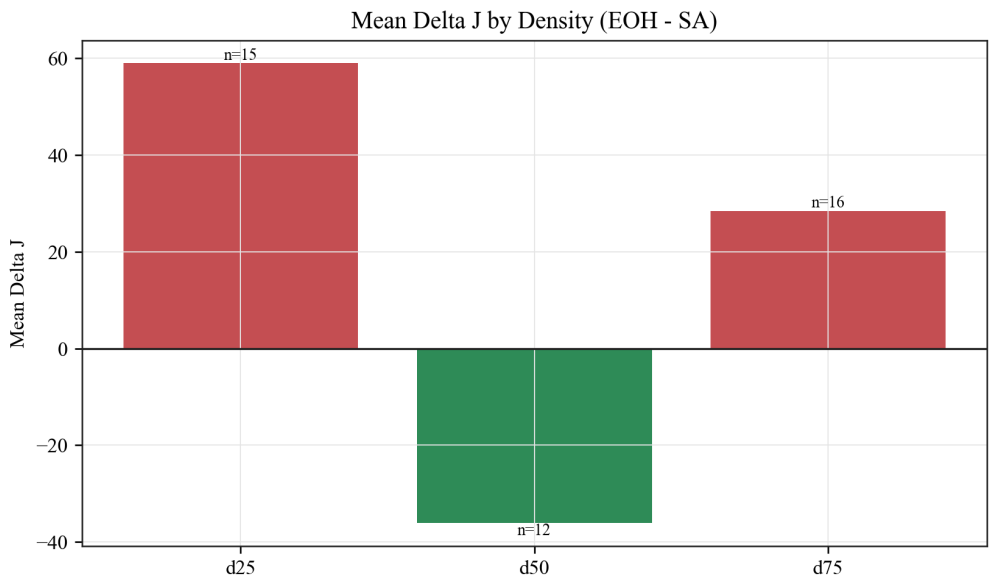

图 4: 不同动态密度下平均 ΔJ。负值表示 EOH 平均优于 SA。

样强调不同动态比例和超参数会改变算法表现；本文的结果进一步说明，LLM 进化出的代码也具有类似的场景敏感性。

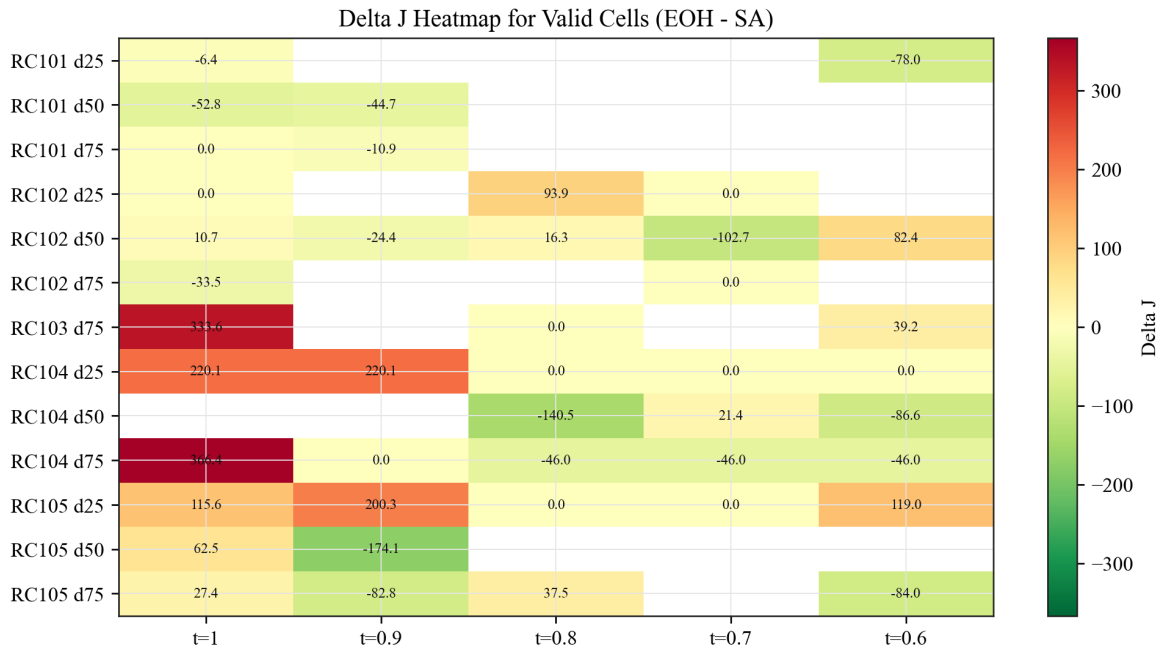

图 5: 所有有效 cell 的 ΔJ 热力图。绿色区域代表 EOH 改善，红色区域代表变差。

6.5 质量与响应时间的权衡

图 6 和图 7 分别比较 SA 与 EOH 的 J 和 Res.。如果只看 J，部分 EOH 候选确实能明显降低成本；但从 Res. 散点和质量–时间权衡图看，不少改善伴随响应时间增加。这一点与参考论文的评价思想完全一致：实时动态调度不能只追求最低 J，还要把响应速度作为核心指标。因此，本文没有把 EOH 结果归结为单一胜率，而是使用 Res./J 双指标解释其适用边界。

6.6 候选质量与 guard 的作用

图 9 展示不同 outcome 下通过 guard 的候选数量。该图的意义在于说明：有效候选数量本身并不保证结果改善，但候选能否通过 guard 决定了它是否有资格进入论文主表。没有 guard 时，raw-best 可能被异常极小值占据；有 guard 后，论文表格展示的是更可信的 filtered-best。这也是本文相较于直接运行原版 EOH 的关键改进。

7 Repeat Validation

由于 LLM 生成和进化搜索具有随机性，单次运行的 improved cell 不能直接视为稳定结论。我们从初始主表中选择八个高价值 cell 进行 repeat validation，每个 cell 重新运行两次。结果表明，RC102 和 RC104 中部分初始改善在 repeat 中退化为持平或变差；相对更值得保留的证据集中在 RC105 的 $d50, t = 0.9$ 与 $d75, t = 0.6/0.9$。在 8 个 repeat cell 中，有 3 个至少出现一次 improved，有 3 个 mean ΔJ 为负。

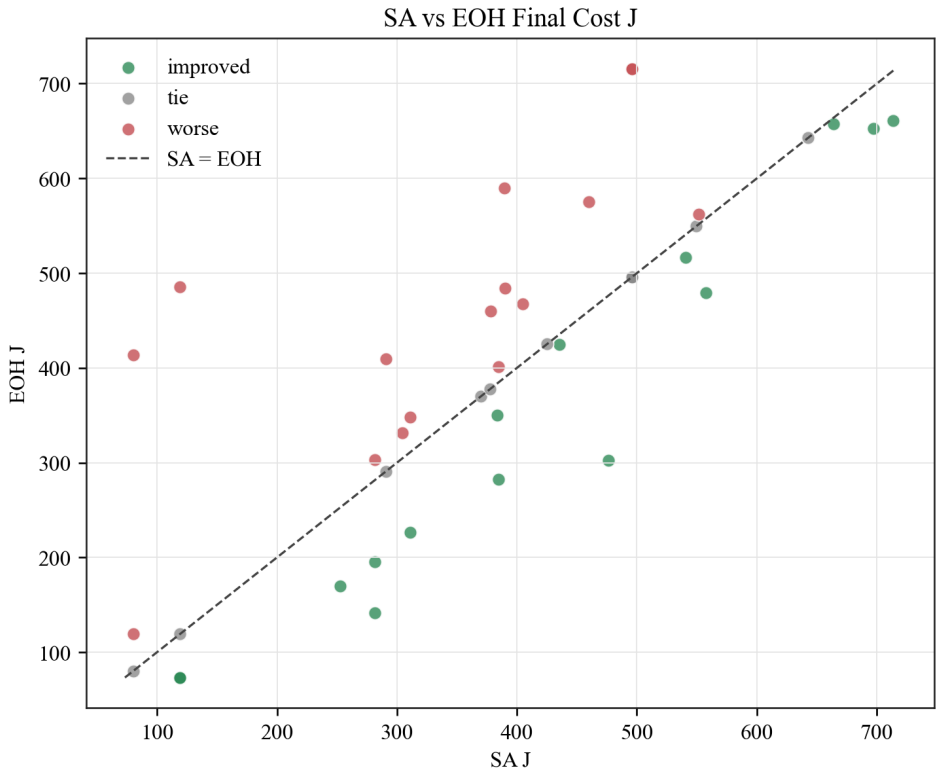

图 6: SA 与 EOH 的最终成本 J 散点图。虚线表示二者相等。

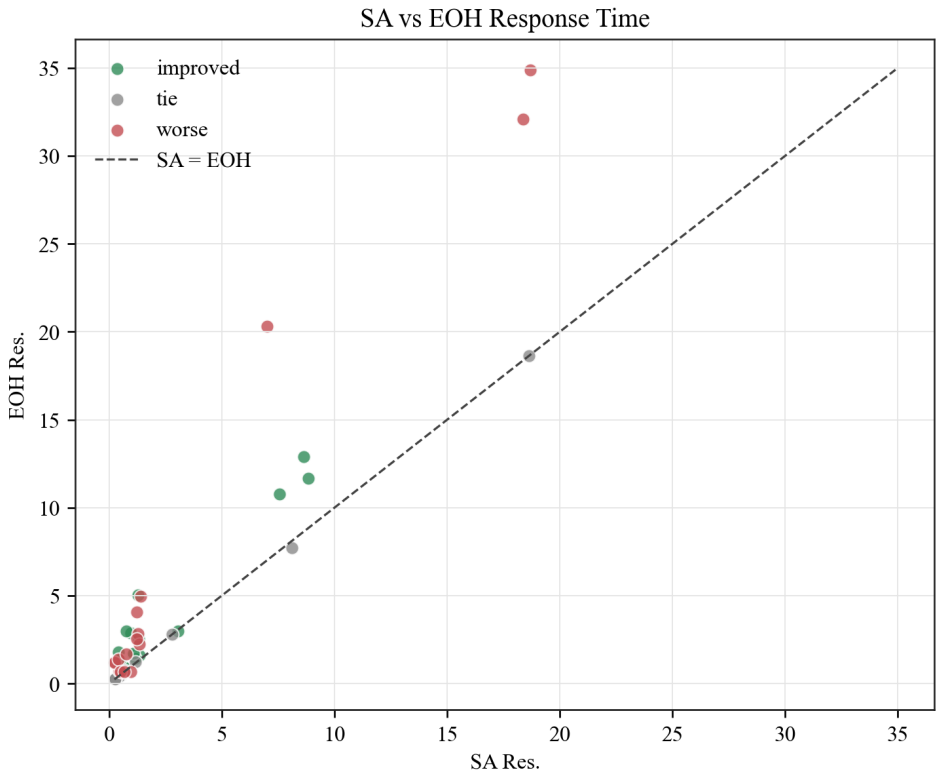

图 7: SA 与 EOH 的响应时间 Res. 散点图。虚线表示二者相等。

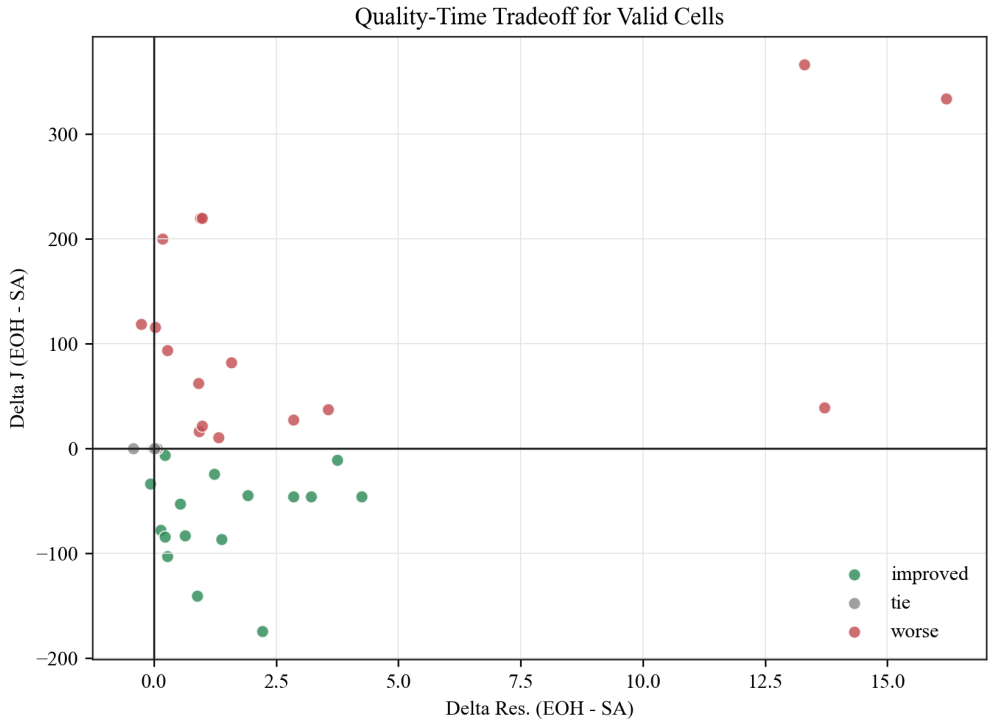

图 8: 有效 cell 中 $\Delta Res.$ 与 ΔJ 的权衡关系。左下区域代表 EOH 同时更快且质量更好，右下区域代表质量更好但更慢。

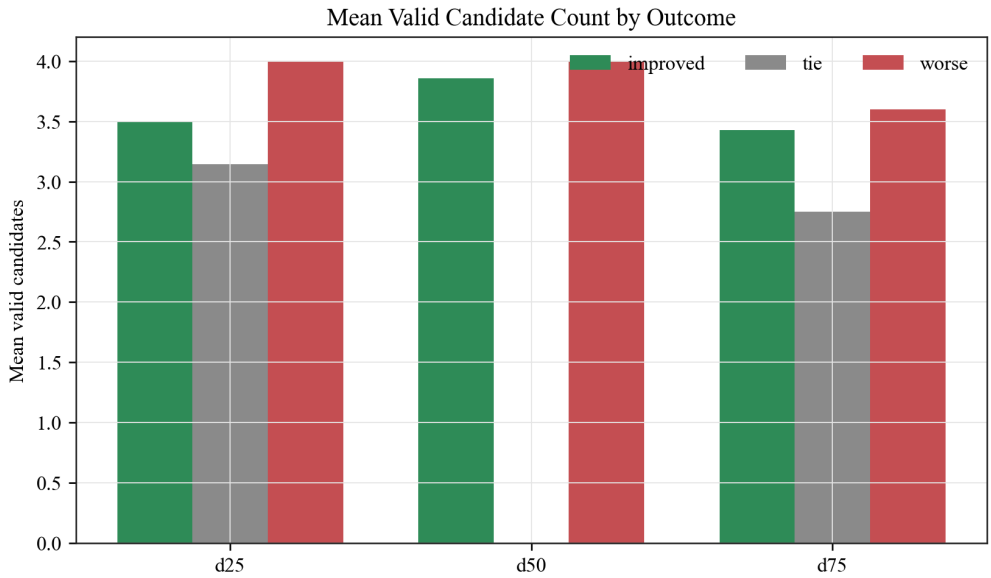

图 9: 不同 outcome 下平均有效候选数量。

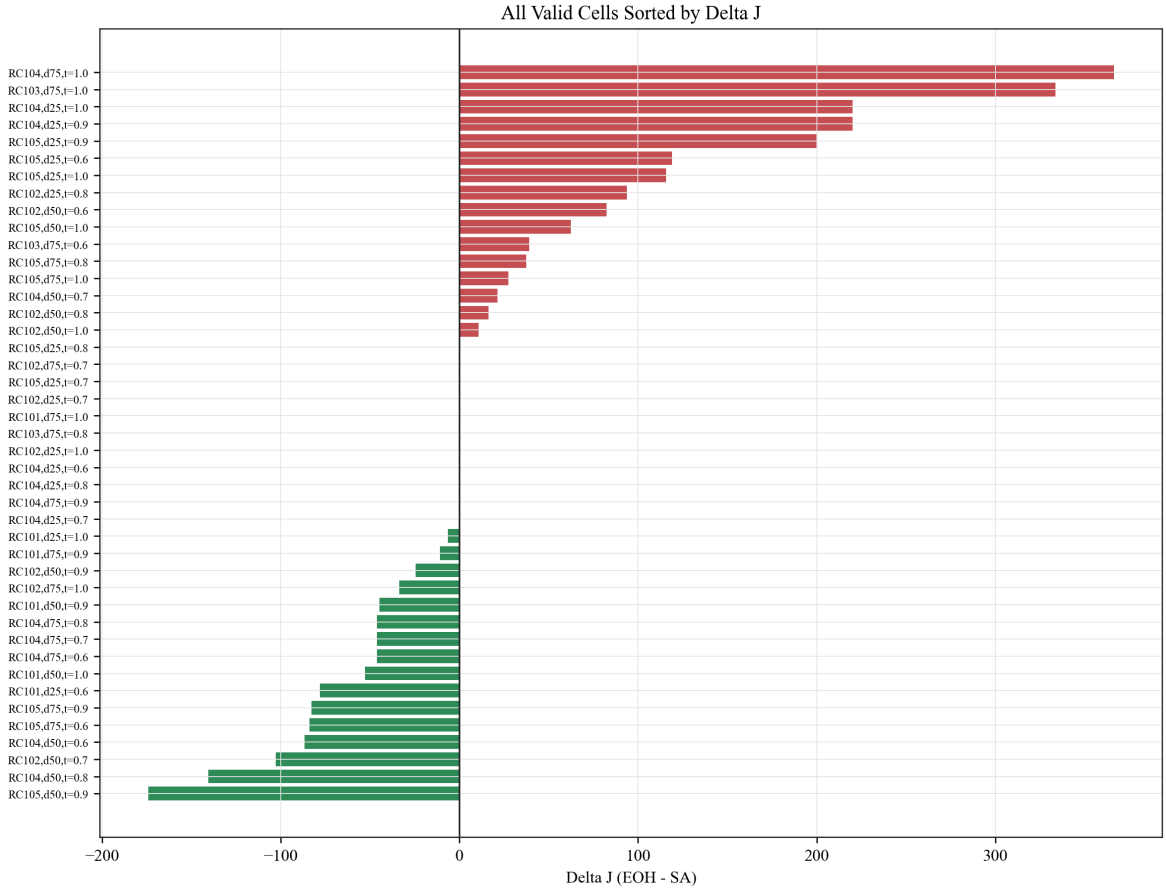

图 10: 所有有效 cell 按 ΔJ 排序后的条形图。

这一结果加强了本文的保守表述：EOH-Go 可以发现有价值的插入启发式，但这些启发式具有场景敏感和随机不稳定性，需要 repeat validation 支撑。对于论文写作而言，repeat 表不应被看作削弱结果，而是提升可信度的必要环节。

Table 2 Repeat validation of selected Guarded EOH cells

Instance	d	t	Runs	Outcome counts				ΔJ over repeats	
				Imp.	Tie	Worse	Excl.	Mean	Best
RC102	d50	0.7	2	0	1	1	0	+358.54	0.00
RC102	d75	1.0	2	0	1	0	1	0.00	0.00
RC104	d50	0.8	2	0	1	1	0	+10.70	0.00
RC104	d50	0.6	2	0	0	2	0	+138.02	+49.62
RC104	d75	0.8	2	0	2	0	0	0.00	0.00
RC105	d50	0.9	2	1	1	0	0	-17.57	-35.14
RC105	d75	0.9	2	1	1	0	0	-13.00	-25.99
RC105	d75	0.6	2	1	1	0	0	-42.00	-84.00

Note: Repeat data are reruns of selected cells from the cleaned main grid. Mean and Best use Delta J = EOH J - SA J; negative values favor EOH.

图 11: 所选 cell 的 repeat validation 表。Mean 与 Best 均使用 $\Delta J = J_{EOH} - J_{SA}$。

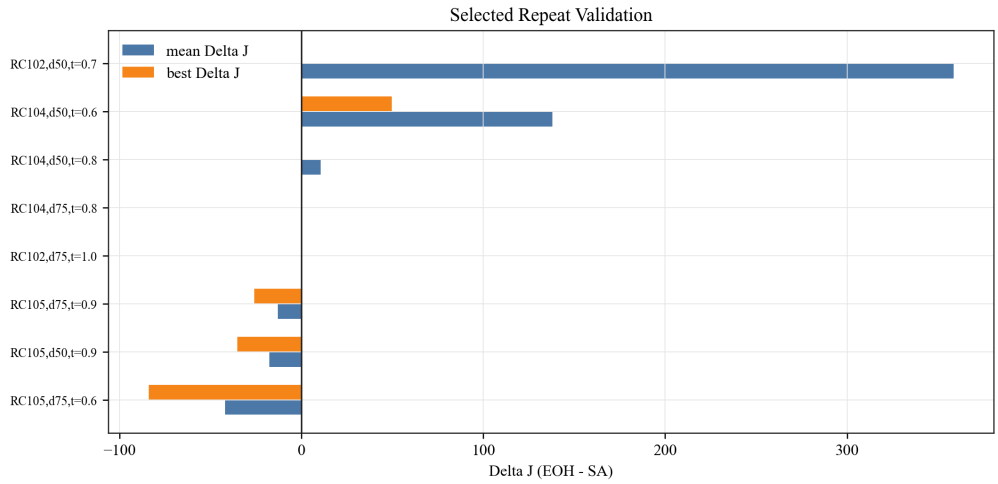

图 12: repeat validation 中 mean ΔJ 与 best ΔJ 的图示。

8 讨论

8.1 与参考实时路由论文的关系

本文不是要替代参考论文中的实时路由系统，而是在其问题设定和评价方式上扩展一个新的自动启发式设计层。参考论文证明了实时路由求解器可以通过管理模块、自适应内存、插入任务和优化任务在响应速度与路线质量之间取得平衡；本文则研究插入启发式本身是否可以由 LLM-driven EOH 自动生成。两者的关系可以理解为：参考论文提供动态调度问题框架和 Res./J 评价标准，本文在该框架中引入自动代码进化机制。

8.2 为什么不能声称 EOH 全面优于 SA

从主表看，EOH 在 16 个有效 cell 中改善 SA，但也在 16 个 cell 中变差。这种结果不适合包装成单一胜率故事。更准确的解释是，EOH 在特定动态密度和到达节奏下能发现局部有效的插入策略，但当前搜索深度、候选数量和 guard 强度还不足以保证跨实例稳定优势。特别是 repeat validation 显示部分强胜点并不复现，因此论文结论必须保持谨慎。

8.3 guard/filter 的方法论意义

本文最重要的方法论发现是：LLM 代码进化会放大 evaluator 的漏洞。传统启发式比较通常默认算法逻辑固定，评价器只需计算输出质量；而在 EOH-Go 中，候选代码本身可能改变输出结构、跳过插入步骤或触发异常路径。若没有 guard，异常极小值会被误认为算法进步。本文将 suspicious-low、negative output、missing result 等情况显式排除，并保留 raw 记录用于追踪。这使得结果更保守，但也更接近可发表实验。

8.4 有效性边界

当前结论应被理解为一个受控范围内的实验结论。第一，本文的主实验范围限定在 RC101–RC105、$d25/d50/d75$ 和 $t = 1.0$ 到 0.6 的网格上，因此结论主要说明 Guarded EOH-Go 在这一组动态源和到达节奏下的表现。第二，EOH 设置较轻量，generations=1、pop_size=4，目标是验证可执行 Go 启发式进化链路的可行性，而不是证明搜索过程已经充分收敛。第三，repeat validation 的作用是检查初始 improved cell 的稳定性；它显示部分胜点会退化，因此本文只把稳定性较强的 RC105 cell 作为相对更可信的证据，而不把所有单次 improved cell 都视为稳定优势。第四，guard 规则目前采用成本阈值、缺失输出、负值输出和 suspicious-low 过滤，主表结论严格限定在这些过滤规则之下。

9 结论

本文基于参考论文的实时动态取送货路由问题设定，构建了一个 Guarded EOH-Go pipeline，用于自动进化 Go 调度程序中的 `InsertShips` 插入启发式。实验表明，在 RC101–RC105 的动态密度与到达节奏网格中，EOH 能在部分 cell 中改善 SA 的最终成本 J，但也存在持平、变差和被 guard 排除的情况。因此，本文的核心结论不是 “EOH 全面击败 SA”，而是 “在可信评价和过滤机制保护下，LLM-driven EOH 可以作为动态实时路由中场景相关启发式设计工具”。

10 未来展望

本文没有把尚未完成的扩展路线写入方法主线，而是将其作为后续工作。第一，可以把实例范围从 RC101–RC105 扩展到 RC106–RC108，并对 RC105 中相对稳定的胜点继续增加 repeat，以提高统计可信度。第二，可以将订单完整性检查、时间窗检查和车辆状态一致性检查加入 guard，使 filtered-best 不只依赖成本阈值，还能验证候选代码是否完整服务了动态请求。第三，可以引入 ReEvo-style feedback，把 compile fail、timeout、suspicious-low、worse-than-seed 等失败原因转化为下一轮 prompt 的文字反馈，从而让 EOH 的变异方向更稳定。第四，ReAct、多 agent 调

度和后训练机制更适合放在毕业论文或下一阶段系统中探索：它们可以用于自动选择任务、管理候选库、分析失败原因和调度多轮实验，但不属于本文已经完成的最小实现。当前文章应保持主线简洁，突出 Guarded EOH-Go 在实时动态路由启发式代码进化中的可行性和方法论价值。

A 附录：按实例划分的密度–到达节奏热力图

以下图展示每个实例内部 d 与 t 对 EOH 表现的影响。它们主要用于支撑“场景敏感”这一讨论，而不是单独作为全局结论。

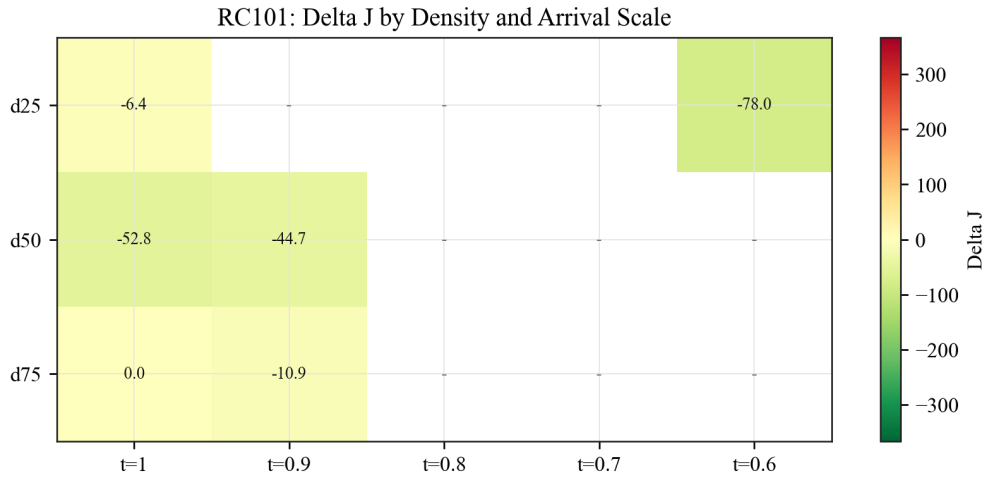

图 13: RC101 的 $d \times t$ 有效 cell ΔJ 热力图。

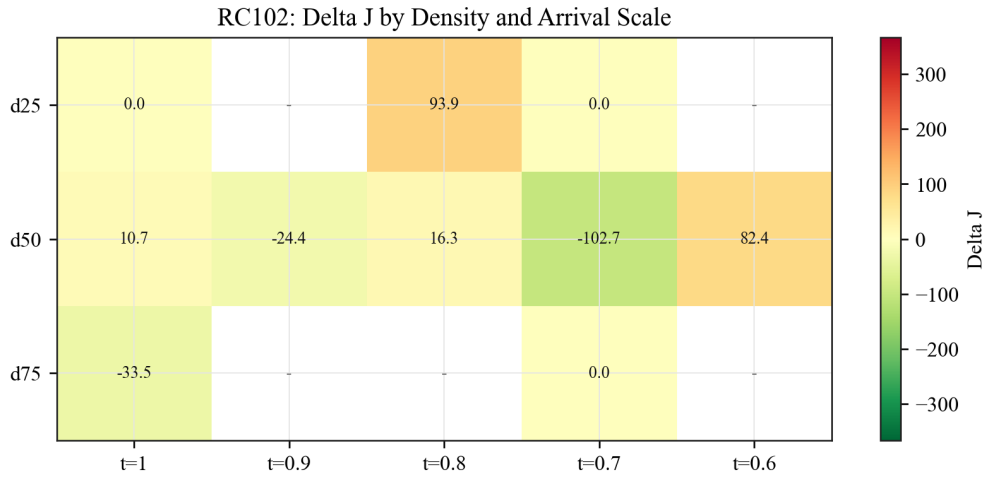

图 14: RC102 的 $d \times t$ 有效 cell ΔJ 热力图。

参考文献

[1] 参考论文：实时路由与并发计算在封闭商业园区配送服务中的应用，本地文件 第二篇论文中文.pdf。

[2] Fei Liu et al. *Evolution of Heuristics: Towards Efficient Automatic Algorithm Design Using Large Language Model*. ICML 2024.

[3] Bernardino Romera-Paredes et al. *Mathematical discoveries from program search with large language models*. Nature, 2023.

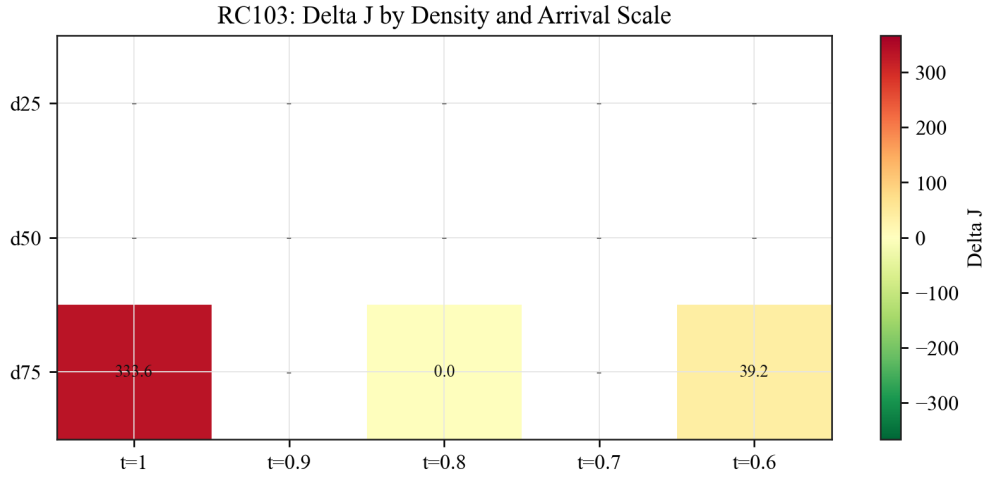

图 15: RC103 的 $d \times t$ 有效 cell ΔJ 热力图。

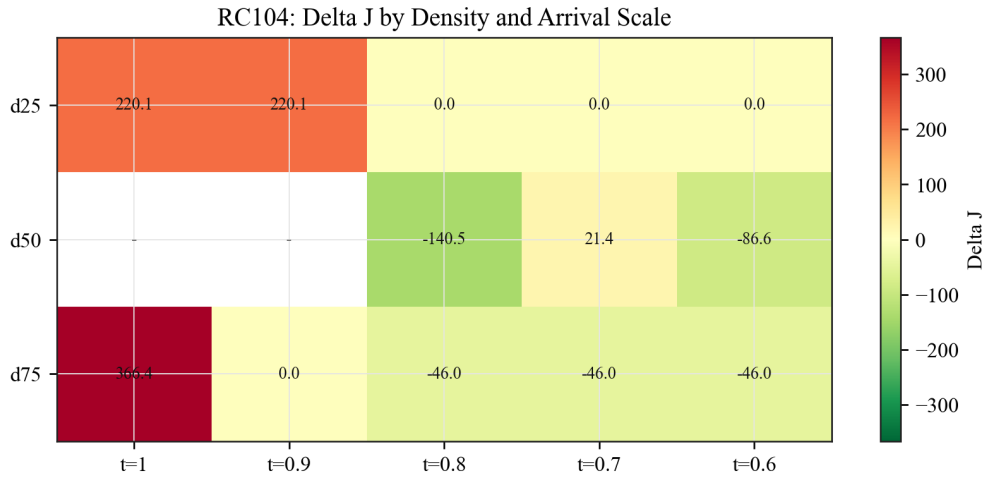

图 16: RC104 的 $d \times t$ 有效 cell ΔJ 热力图。

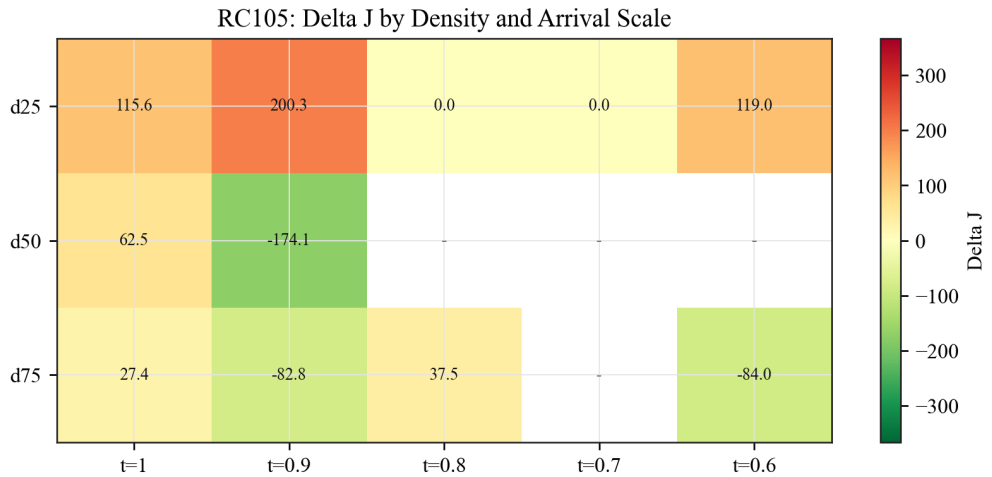

图 17: RC105 的 $d \times t$ 有效 cell ΔJ 热力图。

[4] ReEvo: Reflective Evolution for Large Language Model-guided Optimization.

[5] LLM4AD: A Platform for Large Language Model Assisted Algorithm Design.